UPEN

Career: Engineering in Information Technologies and Digital Innovation

Subject: Object-Oriented Programming

Activity: Justification of use case diagram

Student: Alex Antonio Pinto Cataño

Delivery Date: September 14, 2026

Justification of the use case diagram

Credit purchase and delivery system

1. The problem being modeled

The system manages the credit sales process within a company. For a credit purchase to take place, several parties are involved: the customer who wants to buy, the salesperson who processes the sale, the credit department that approves the transaction, and the logistics department (courier) that delivers the product.

The process of a "credit purchase" is complex and requires that a credit be requested, that automatic payments be set up, that the credit be authorized by the corresponding department, and finally, that the product be delivered.

This use case diagram accurately represents the main functionalities of the system and which users (actors) interact with each of them to achieve the business objective.

2. Why was a use case diagram chosen?

A use case diagram illustrates what the system does *and who* interacts with it, providing a high-level view of the functional requirements. It's the right tool here because the goal is to define the system's boundaries and the responsibilities of each user role. A class diagram shows the data structure but doesn't explain the system's purpose. A sequence diagram shows the order in which different elements communicate, but it would be too detailed if we only want to know the general functions and who participates. A use case diagram, however, answers the question: "What functions does the system offer, and which users have access to them?"

3. The participants: why these and not others

- **Customer: The stick** figure represents the customer because they are an actor, meaning someone external to the system who uses it. They are the final beneficiary of the purchase and the one who provides the information for the credit.
- **Salesperson: This** is the employee who interacts with the public. They interact with the system to register the sale and initiate the credit application process with the customer.
- **Credit Dep Rep: This i**s a specialized role. Their function is exclusive and critical: to evaluate and authorize credit applications. This role must be separate from the salesperson's for control and security.

- **Courier (Messenger/Delivery Person):** The actor responsible for physical logistics. They interact with the delivery system to determine what to deliver and where, marking the culmination of the sales process.

4. Route and justification of each element

The following table explains where each part of the diagram comes from and why it has that shape.

Use Case / Relationship	**Justification (why it exists)**	**Notation used**
Purchase (Credit Purchase)	This is the core functionality of the model. It encompasses the entire process of buying something on credit. It is initiated and managed primarily by the seller.	Oval (Use Case). Solid line (association) from the *Salesperson actor*.
Request credit	To make a purchase on credit, the first logical step is to apply for it. This involves both the customer (who provides the information) and the seller (who enters it).	Oval. Continuous line connected to the *Customer* and *Salesperson actors*.
Automatic Payments	All credit requires a method of repayment. The system must schedule payments automatically once a purchase is completed.	Oval. It has no physical actor connected because the system executes it internally.
Credit Authorization	A loan application is not approved on its own; it requires a human analyst to review the customer's profile and give the green light.	Oval. Solid line connected to the actor *Credit Dep Rep*.
Delivery	Once the sale is finalized, the physical product must reach the customer. The courier uses this functionality to manage their route or confirm delivery.	Oval. Continuous line connected to the *Courier actor*.
Relationships <<include>>	The diagram shows that *Purchase* "includes" the other four use cases. This is because, by business definition, you cannot complete a credit purchase without applying for credit, authorizing it, setting up payments, and delivering the funds. These are mandatory steps of *the cor*e	Dotted arrow with the label <<include>> pointing from the base case (*Purchase*) towards the

	functionality.	included cases.

5. Which cases does the diagram cover and which does it not (and why)

- **Cases that are covered:**
  - The complete and mandatory functional flow to process a credit sale.
  - Separation of roles: who sells, who authorizes, and who delivers.
- **Cases that are NOT covered (and why it's correct that they aren't):**
  - **Cash Sales: The** diagram focuses exclusively on *Credit Purchases*. A cash sale would not require authorizations or automatic payment setup. It would be a different use case.
  - **The exact chronological o**rder: Unlike the sequence diagram, this diagram doesn't tell you whether authorization happens before or after setting up automatic payments; it only tells you th*at both must happen*.
  - **What happens if the loan is rejected: Use c**ase diagrams model the functional goals or intentions. The alternative or error flow (rejection) would be documented in the textual *specification of the u*se case, or in an activity/sequence diagram.

6. How to read the notation (to explain the symbols if asked)

- **Stick figure = Actor:** someone external to the system who uses it (people or roles that operate the software).
- **Named oval = Use case:** A functionality, goal, or service that the system provides.
- **Large rectangle = System boundary:** Everything inside the rectangle is the software we are designing. The actors are outside because they interact with it from the outside.
- **Solid line = Association:** Indicates which actor is communicating with which use case. They do not have an arrow because communication is usually bidirectional (the actor requests something, the system responds).
- **Dotted arrow with <<include>> = Inclusion relationship:** It means "to do A, I must necessarily do B". The arrow always comes from the main use case and points to the use case being included.

7. Possible questions from the teacher and how to answer them

Why did you connect two actors (*Customer* and *Salesperson*) to the same use case (*Request credit*)?

- **A:** Because a use case represents an interaction, and in the credit application both actively participate with the system: the customer provides their personal and financial information, and the seller is the one who guides the process and registers the application on the platform.

Why did you use <<include>> and not <<extend>>?

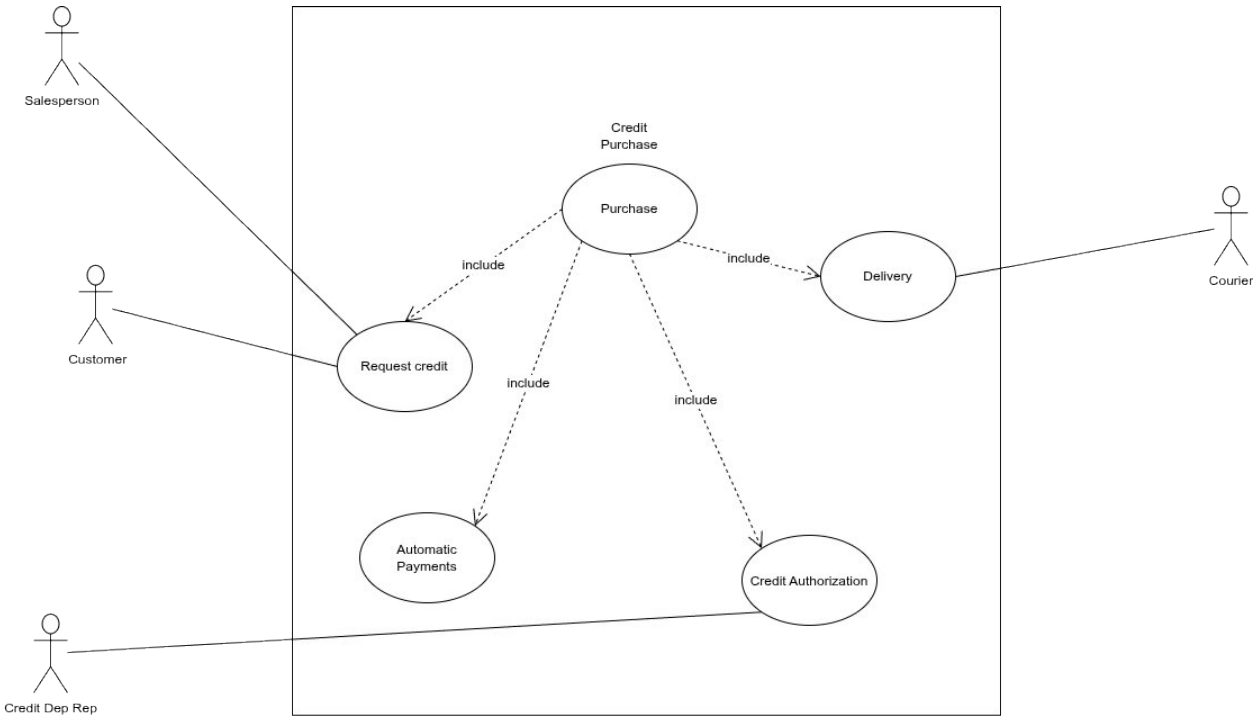
- **A:** Because the diagram assumes a credit purchase scenario. In this context, authorizing the credit, scheduling payments, and delivering the product are not optional steps (<<extend>>), they are mandatory components without which the main goal (*Purchase*) would be incomplete.

Why is the *Automatic Payments* use case not connected to any stick figure (actor)?

- **A:** Because it's a task the system performs autonomously (batch) or as a result of a sale, without a user having to manually press buttons in that specific module. The system automatically schedules payments.

Is the *courier* part of the system?

- **A:** No. Since it's a stick figure, in UML that's the standard notation for an actor: someone external to the system who uses it. The *Courier* is a physical employee who uses the system's delivery module; it's not a software function.



Conclusion

Creating this diagram was like drawing a map of a company's main tasks, where the biggest challenge was organizing who does what and what things are strictly necessary to sell something on credit:

- **Clearly defining roles:** Initially, the challenge was identifying all the people involved. Instead of assigning a single, generic employee, we decided that for organization and security, we needed to clearly separate the salesperson, the credit approval employee, and the delivery person. We also included the customer because they actively participate by providing their information to initiate the process.

- **Identifying the mandatory steps:** Another major challenge was deciding how to connect the different store activities. We concluded that to complete a credit sale, applying for the loan, obtaining approval, scheduling payments, and delivering the product are not steps that can be omitted. They are absolutely mandatory requirements for the sale to function, so we decided to link them as essential parts of the main purchase.

- **Recognizing automated work: It was** very interesting to realize that not all tasks require a human to be pressing buttons. For example, the platform handles the scheduling of automatic payments on its own. The solution to this design challenge was to draw that task on its own, without connecting it to any person, thus representing that the software takes care of it independently.

- **Focusing on the "what" rather than the "when":** Ultimately, we had to resist the temptation to try to show the exact chronological order in which things happen. We decided to focus solely on showing the functions the system offers and who has access to them, intentionally leaving out scenarios like cash purchases or what happens if a loan is rejected. This helped us keep the diagram clean, straightforward, and easy for anyone to understand.